

LES ORGANISATION DES FICHIERS

(Algorithmes mis en oeuvre par le SGBD/SGF pour mémoriser et retrouver les données)

- ❑ Tas (Heap)
- ❑ Triée
- ❑ Séquentielle Indexée
- ❑ B-arbre
- ❑ H-Code

◆ **Notion d'Organisation :**

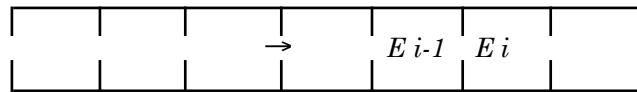
La façon dont les enregistrements sont rangés dans un fichier.

◆ **Méthode d'Accès :**

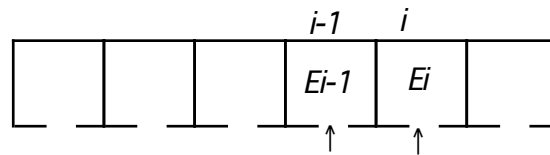
C'est la façon, en fonction de l'organisation, dont les enregistrements sont accédés.

◆ Accès séquentiel

Pour aller à l'enregistrement E_i passer par l'enregistrement E_{i-1}



◆ Accès direct



→ Clé $\in$ enregistrement (e.g. N°CIN, N°Matr ...)

→ Clé $\notin$ enregistrement (e.g. Rang dans le fichier, *rid*)

❑ Organisation Tas (en vrac)

Les enregistrements sont rangés au fur et à mesure, sans ordre particulier.

Tomates	2
Poireaux	5
Pommes	9
Carottes	3.5
Oranges	2
Bananes	9
...	

- Avantage : Simplicité de gestion (insertion / suppression / consultation)
- Inconvénient : Recherche non performante (séquentielle)
- *Convient pour fichiers de petite taille*

❑ Organisation Tas (en vrac)

- **Recherche :**

Ouvrir (fichier);
lire (fichier) → enr;
Tant que (non fin-fichier) ***faire***
traiter (enr);
lire (fichier) → enr;
fait;
fermer (fichier)

- **Insertion :** Au bout du fichier

- **Suppression :** Logique (indicateur d'effacement)

Physique (remplacement par le dernier)

Ou recopie du fichier (sans les enregistrements effacés)

□ Organisation Triée

Les Enregistrements sont rangés par ordre croissant (ou décroissant) d'une *clé*.

Bananes	9
Carottes	3.5
Oranges	2
Pommes	9
Poireaux	5
Tomates	2
...	

- Avantage : Possibilité recherche plus rapide (e.g. dichotomique).
- Inconvénient : Mise à jour plus contraignantes (insertion à la bonne place).

❑ Organisation séquentielle indexée (ISAM *indexed sequential access method*)

- Le fichier est trié par rapport à une clé.
- Les enregistrements sont groupés par blocs.
- Accès direct au bloc contenant un enregistrement.

Fichier index

<i>Clé</i>	<i>pid</i>
Bananes	1
Choux	2
Mandarines	3
Oranges	4
Tomates	5

Les enregistrements de clé *C* telle que :

"Mandarines" < C ≤ "Oranges"

se trouvent dans le bloc 4

Fichier indexé

1	Amandes	80
	Ananas	12
	Bananes	9
2	Betteraves	2
	Carottes	3.5
	Choux	1
3	Endives	7
	Laitue	2.5
	Mandarines	3
4	Navets	3
	Oignons	4.5
	Oranges	2
5	Poireaux	5.5
	Pommes	9
	Tomates	2

Remarque : Ici, blocs remplis à 100%

❑ Organisation séquentielle indexée

- **Recherche :** En deux étapes. Soit à chercher une clé C :
 - 1- *Chercher dans le fichier index un rang i tel que:*
$$Clé_{i-1} < C \leq Clé_i$$
On a alors, pid_i qui est le n° de bloc où C devrait se trouver
 - 2- *Chercher dans le fichier indexé dans le bloc pid_i l'information relative à C .*
Si C n'existe pas, C n'appartient pas au fichier...
(Voir plus bas)
- **Exemple :** $C = \text{"Navets"}$
 - 1- $\text{"Mandarines"} < \text{"Navets"} \leq \text{"Oranges"}$
$$\Rightarrow \text{Bloc 4}$$
 - 2- Bloc 4 du fichier on a l'enregistrement : $\langle \text{Navets, prix} = 3 \rangle$

□ Organisation séquentielle indexée

- **Insertion :** Soit à insérer une clé C :
 - 1- *Chercher dans le fichier index le n° de bloc où C devrait se trouver*
 - 2- *Si il existe de la place dans le bloc, **alors** y mettre l'enregistrement à insérer. Sinon insérer l'enregistrement dans la zone de débordement.*

La zone de débordement est un ensemble de blocs, en générale en bout de fichier, et qui contient tous les enregistrements qui n'ont pas trouvé place dans leur bloc.

- **Suppression :** – Suppression logique.
 - Mise à jour de l'index (si entrée index concernée, e.g. supprimer “Choux”)
- **Création :** – Dans l'ordre croissant des clés. Index remplis au fur et à mesure
 - Remplissage des blocs à $n\%$ ($n < 100$).

❑ Organisation séquentielle indexée

- **Réorganisation du Fichier :**
 - Si le débordement est trop important, il devient pénalisant.

<u>Index</u>		<u>Fichier Indexé</u>	
Bananes	1	1	Amandes 80
Choux	2		Ananas 12
Mandarines	3		Bananes 9
Zone de débordement		2	Betteraves 2
			Carottes 3.5
			Choux 1
		3	Endives 7
			Laitue 2.5
			Mandarines 3
			...
			Kiwis 3
			Oranges 2
			Navets 3
			Artichauts 6.3
			Céleris 4.5
			Fenouils 2
			...

Fichier avec débordement

❑ Organisation séquentielle indexée

- **Réorganisation du Fichier :**

- Le fichier ISAM est automatiquement recréé avec un nouvel index.

Nouvel Index

Ananas	1
Bananes	2
Carottes	3
Choux	4
Kiwis	5
Laitue	6
Oranges	7

1	Amandes	80
	Ananas	12
	////////////////////////////////	
2	Artichauts	6.3
	Bananes	9
	////////////////////////////////	
3	Betteraves	2
	Carottes	3.5
	////////////////////////////////	
4	Céleris	4.5
	Choux	1
	////////////////////////////////	
5	Endives	7
	Kiwis	3
	////////////////////////////////	
6	Fenouils	2
	Laitue	2.5
	////////////////////////////////	
7	Mandarines	3
	Navets	3
	Oranges	2

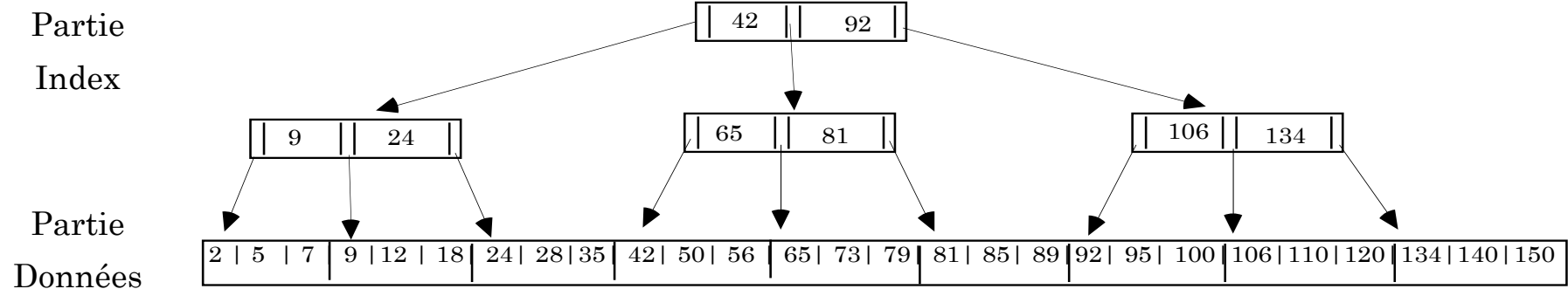
Fichier réorganisé (//////////////////////////////// place libre)

□ Les B-arbres

- Bayer 68
- Index multi-niveau
- Arbre Balancé
- ~ Organisation VSAM (IBM)
- Très performante, très couramment utilisée.

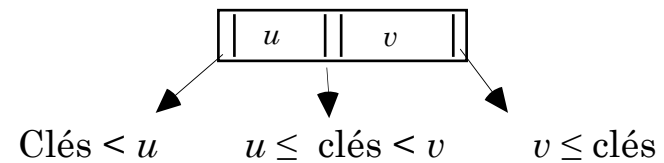
□ Les B-arbres

- Variante de *D. E. KNUTH*



Seule les clés sont montrées.

- Rangement clés croissantes
- Valeurs des nœuds $\in$ au fichier
- Nœud tel que :



□ Les B-arbres

- **Recherche :** Soit C la clé à chercher. N représente un noeud:

$N = \text{racine}$;

Répéter

si $C < N.u$ **alors** $N = N.\text{Fils}G$;

si $N.u \leq C < N.v$ **alors** $N = N.\text{Fils}M$;

si $N.v \leq C$ **alors** $N = N.\text{Fils}D$;

Jusqu'à N dans partie données.

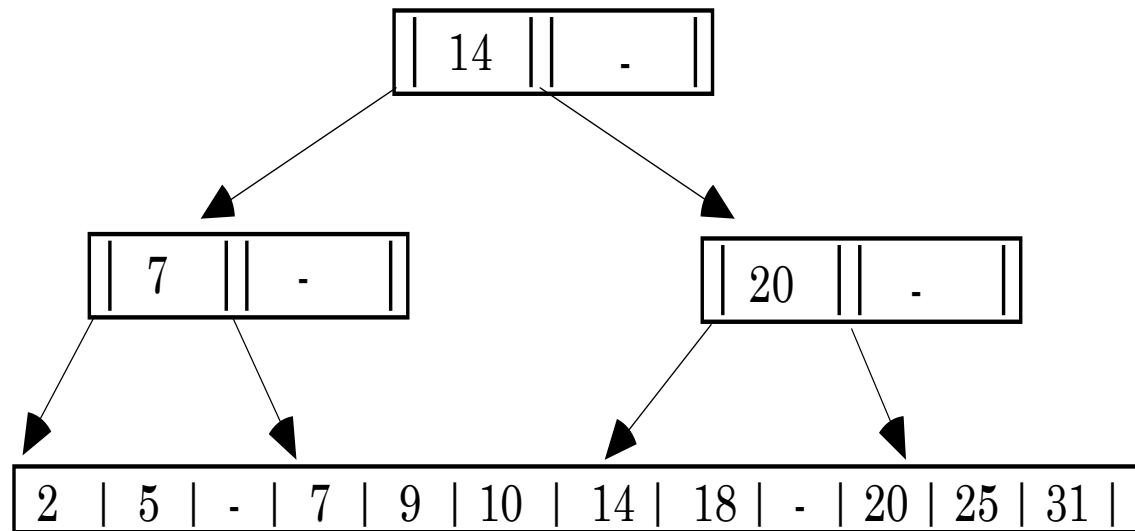
Chercher C dans le bloc ainsi trouvé. Si n'existe pas alors $C \notin$ au fichier

- **Exercice :** Montrer le chemin pour trouver Les clés

24, 85, 99

□ Les B-arbres

- **Insertion :** Arbre croît progressivement



Début d'un B-arbre

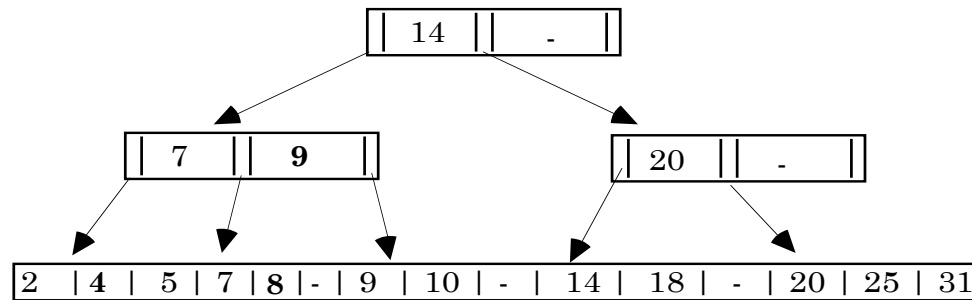
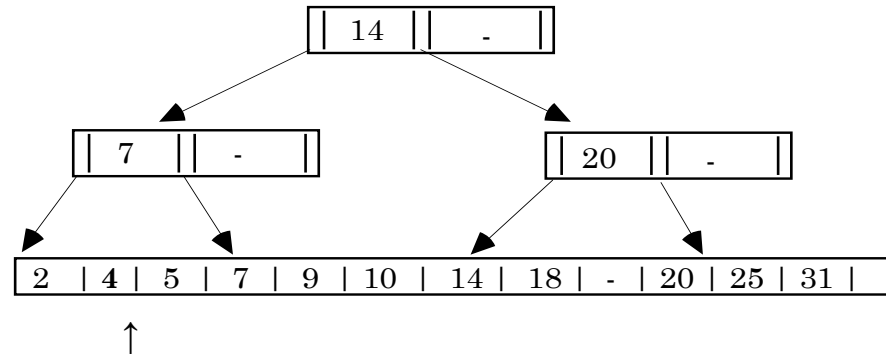
Contrainte : Garder l'arbre balancé.

Ordre d'un B-arbre : ordre n si maximum $2n$ valeurs par nœud, minimum n .

Ici, b-arbre d'ordre 1.

□ Les B-arbres

- **Exemple :** Ajouter 4 (*place existe*) ensuite 8 (*dédoubler bloc*)



Au pire des cas, on rajoute une racine, i.e. un niveau supplémentaire

□ Organisation H-code

- Dite aussi adressage calculé ou adressage dispersé.
- A chaque entrée, est associée une adresse (n° bloc) de rangement calculée en fonction d'une clé.
- La fonction de calcul s'appelle fonction H-code :

$$\begin{aligned} h : \text{Entrée} &\mapsto [0 .. N] \\ x &\mapsto h(x) \end{aligned}$$

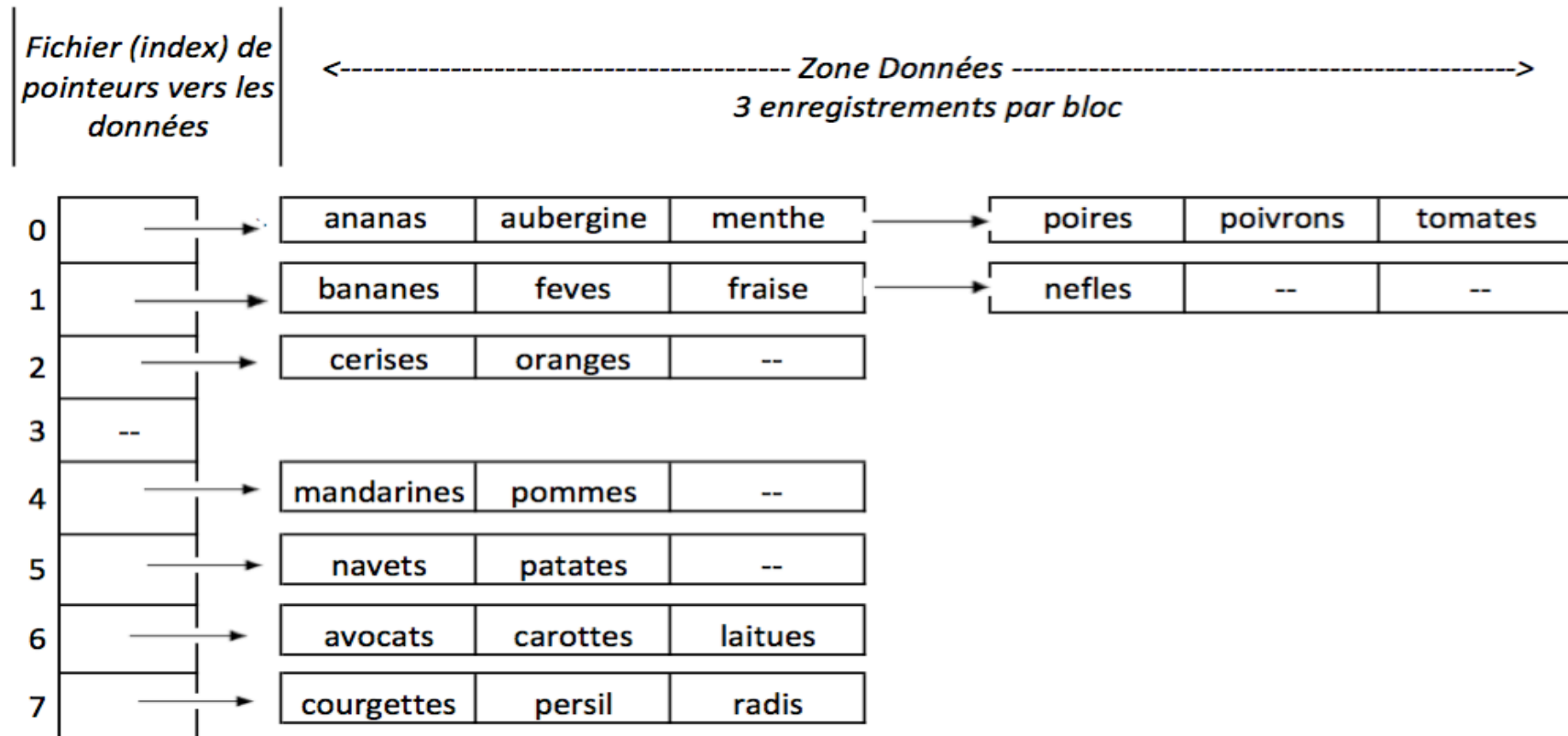
- Exemple de fonction $h(x)$: la somme (module $N+1$) des valeurs binaires des n premiers octets de la représentation mémoire de x .

$$h(\text{"tomate"}) = [\text{ascii}(\text{'t'}) + \text{ascii}(\text{'o'}) + \text{ascii}(\text{'m'})] \bmod (N+1) = 336 \bmod (N+1).$$

- Collision : Quand deux entrées ont le même H-Code. ("ananas" et "tomates", $n=3$, $N=8$)
- Une bonne fonction H-Code est une fonction qui limite les collisions, i.e. qui répartit bien (disperse) les entrées dans l'espace de stockage. Idéalement injective.

Exemple de fichier avec une organisation H-Code

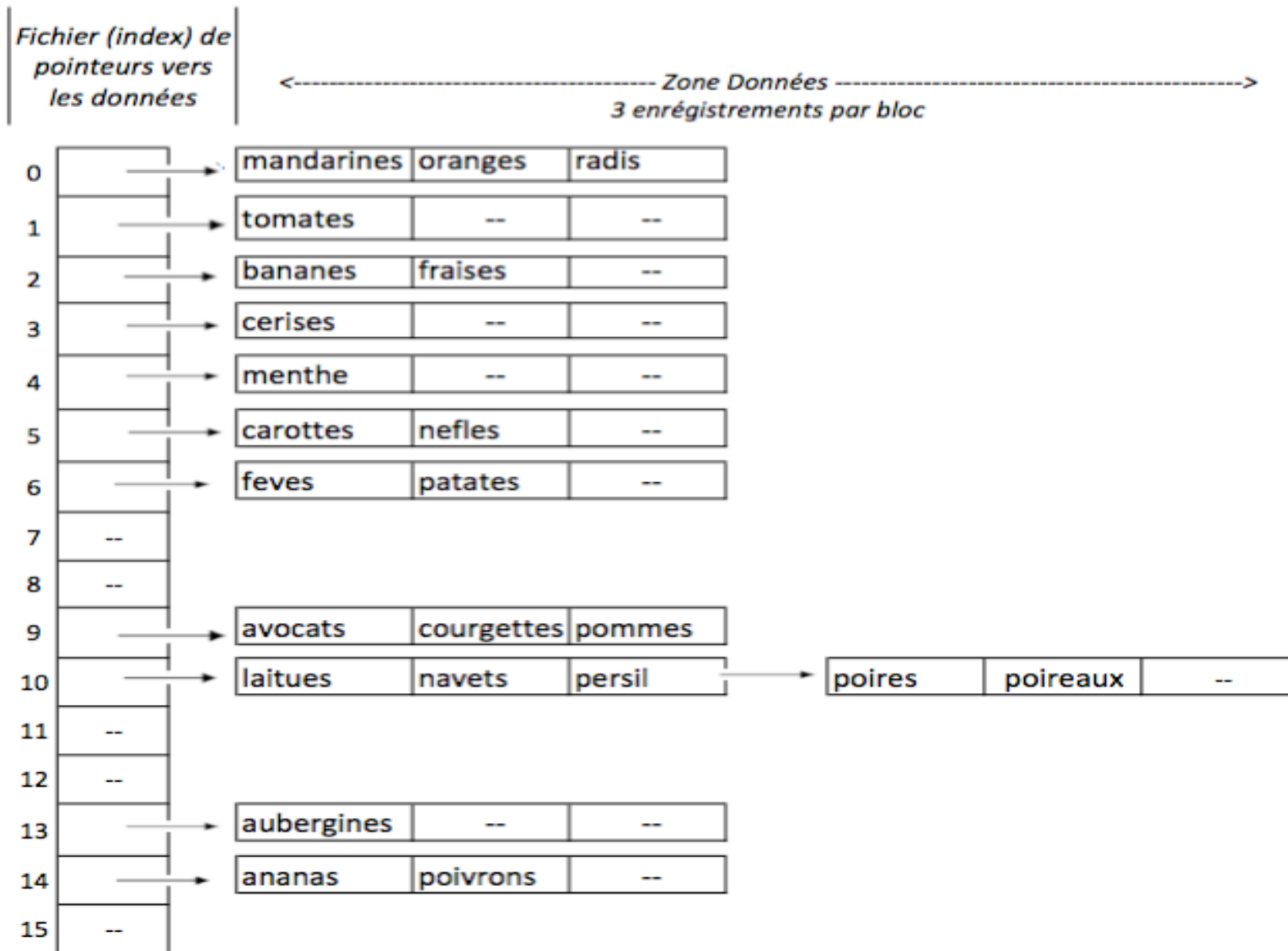
$$h("tomates") = ('t' + 'o' + 'm') \bmod 8 = (116 + 111 + 109) \bmod 8 = 336 \bmod 8 = 0$$



On garantit qu'au plus 2 blocs seront parcourus pour une recherche.

Insertion de poireaux → réorganisation : N est doublé et on calcule h sur les 4 premiers $(n+1)$ octets.

$$h(\text{"tomates"}) = ('t' + 'o' + 'm' + 'a') \bmod 16 = (116 + 111 + 109 + 97) \bmod 16 = 433 \bmod 16 = 1$$



❑ Les index secondaires

Index sur la colonne Ville

Ville	rid
Agadir	4
Casa	2
Casa	6
Casa	8
Fes	7
Rabat	5
Rabat	1
Tanger	3
Tanger	9

Table indexée

	Numéro	Nom	Salaire	Ville	Dept
1	E1	Ali	8000.00	Rabat	D1
2	E2	Ahmed	6000.00	Casa	D1
3	E3	Fatima	7000.00	Tanger	D2
4	E4	Said	5000.00	Agadir	D3
5	E5	Amina	8000.00	Rabat	D3
6	E6	Aziz	7000.00	Casa	D1
7	E7	Amine	5000.00	Fes	D2
8	E8	Ahmed	4000.00	Casa	D4
9	E9	Fatima	7000.00	Tanger	D4

- ✓ Accès très accéléré aux lignes à partir du nom d'une ville

```
SELECT ... FROM ... WHERE Ville= 'Rabat'
```

- ✓ L'index est **trié** (ordre croissant ici)
- ✓ Pas d'ordre particulier pour la table indexée
- ✓ On peut avoir plusieurs index sur la même table

Table indexée

	Numéro	Nom	Salaire	Ville	Dept
1	E1	Ali	8000.00	Rabat	D1
2	E2	Ahmed	6000.00	Casa	D1
3	E3	Fatima	7000.00	Tanger	D2
4	E4	Said	5000.00	Agadir	D3
5	E5	Amina	8000.00	Rabat	D3
6	E6	Aziz	7000.00	Casa	D1
7	E7	Amine	5000.00	Fes	D2
8	E8	Ahmed	4000.00	Casa	D4
9	E9	Fatima	7000.00	Tanger	D4

Index sur colonne Salaire

<i>rid</i>	Salaire
8	4000
7	5000
4	5000
2	6000
3	7000
6	7000
9	7000
1	8000
5	8000

- ✓ Parfait pour `SELECT ... FROM ... WHERE Salaire > 5000`
- ✓ cf. CREATE INDEX de **SQL**
- ✓ Indexes utilisés par le SGBD pour optimiser les requêtes. Accélérer les accès ou les jointures.